

گزارش کار پروژه نظریه زبان ها و ماشین

آرش امیری

تیر ۱۴۰۵

فهرست مطالب

۲	۱ ماشین تورینگ
۲	۱.۱ پیاده سازی ماشین تورینگ
۲	۱.۱.۱ پیاده سازی و تست
۳	۲.۱.۱ فهرست کردن state ها در اسکریپت های تست
۴	۳.۱.۱ نمایش configuration ماشین به ازای هر انتقال
۵	۴.۱.۱ توصیف عملکرد ماشین های تورینگ نمونه
۷	۲.۱ توابع در ماشین تورینگ
۷	۱.۲.۱ جمع کننده ی یک یکی
۸	۲.۲.۱ ضرب کننده ی یک یکی
۱۰	۳.۱ مسئله بیزی بیور
	۱.۳.۱ چرا پیدا کردن برنامه های بیدستر پرتلاش کار ساده ای نیست و به
۱۰	عنوان یک مسئله غیر قابل حل شناخته میشود؟
۱۰	۲.۳.۱ تبدیل نوار ماشین تورینگ از یک طرف متناهی به نامتناهی
	۳.۳.۱ چرا استفاده از ژنراتور های پایتون در شبیه سازی ماشین های تورینگ
۱۰	سودمند است؟
۱۱	۴.۳.۱ اجرای برنامه ی دو کارتی بیزی بیور
۱۲	۵.۳.۱ بیزی بیور ۳ و ۴ کارته
۱۴	۶.۳.۱ بیزی بیور ۵ کارته
۱۴	۷.۳.۱ بررسی بهترین بودن برنامه های قبلی
۱۶	۲ اتوماتا های سلولی
۱۶	۱.۲ بازی زندگی کانوی
۱۶	۱.۱.۲ پیاده سازی قوانین چهار گانه ی بازی زندگی
۱۶	۲.۱.۲ گلاپدر و اهمیت آن در GOL
۱۷	۳.۱.۲ پیاده سازی پارسر فایل های الگو
۱۹	۴.۱.۲ بهینه سازی محاسباتی با کانولوشن
۱۹	۲.۲ مورچه ی لانگتون
۱۹	۱.۲.۲ پیاده سازی پایه ی مورچه
۲۰	۲.۲.۲ شبیه سازی مورچه ی لانگتون و اثبات سناریو ها
۲۱	۳.۲.۲ بهبود مورچه با پشتیبانی چند رنگ
۲۳	۳.۲ گیت های منطقی در بازی زندگی
۲۳	۱.۳.۲ گیت اند
۲۴	۲.۳.۲ گیت نات

۱ ماشین تورینگ

۱.۱ پیاده سازی ماشین تورینگ

۱.۱.۱ پیاده سازی و تست

ابتدا کانستراکتور کلاس تورینگ ماشین خود را میسازیم.

```
1 def __init__(self, transitions, start_state='q0', accept_state='qa',
2   ↪ reject_state='qr', blank_symbol=''):
3     self.transitions = transitions
4     self.start_state = start_state
5     self.accept_state = accept_state
6     self.reject_state = reject_state
7     self.blank_symbol = blank_symbol
```

- transitions : همان ترنزیشن رول ها هستند که به صورت دیکشنری خواهند بود
- start_state : استیت شروع ماشین که به طور پیشفرض q0 است
- accept_state : فاینال استیت را مشخص میکند که به طور پیشفرض qa است
- reject_state : استیت ریجکت را مشخص میکند که هر وقت در این استیت رفتیم ماشین ریجکت خواهد کرد و به طور پیشفرض qr است
- blank_symbol : علامت خانه های خالی در نوار را مشخص میکند که به صورت پیشفرض یک رشته خالی است

حالا متد run را مینویسیم

```
1 def run(self, input_):
2     symbols = list(input_)
3     right_hand_side = []
4     left_hand_side = []
5     if not symbols:
6         current_symbol = self.blank_symbol
7     else:
8         current_symbol = symbols[0]
9         right_hand_side = symbols[1:]
10
11     state = self.start_state
12     while True:
13         configuration = {
14             "state" : state,
15             "left_hand_side" : left_hand_side[:],
16             "symbol" : current_symbol,
17             "right_hand_side" : right_hand_side[:]
18         }
19
20     if state == self.accept_state:
```

```

۲۱         yield ('Accept', configuration)
۲۲         break
۲۳     elif state == self.reject_state:
۲۴         yield ('Reject', configuration)
۲۵         break
۲۶     else:
۲۷         yield (None, configuration)
۲۸
۲۹     if (state, current_symbol) not in self.transitions:
۳۰         state = self.reject_state
۳۱         continue
۳۲
۳۳     to_exe = self.transitions[(state, current_symbol)]
۳۴
۳۵     state = to_exe[0]
۳۶     if to_exe[2] == 'R':
۳۷         left_hand_side.insert(0, to_exe[1])
۳۸         if right_hand_side:
۳۹             current_symbol = right_hand_side.pop(0)
۴۰         else:
۴۱             current_symbol = self.blank_symbol
۴۲     elif to_exe[2] == 'L':
۴۳         right_hand_side.insert(0, to_exe[1])
۴۴         if left_hand_side:
۴۵             current_symbol = left_hand_side.pop(0)
۴۶         else:
۴۷             logging.warning("Crossed left boundary of
→ singly-infinite tape")
۴۸             state = self.reject_state
۴۹             continue
۵۰     else:
۵۱         raise ValueError(f"Invalid direction: {to_exe[2]}")

```

این متد یک ورودی میگیرد که همان رشته ورودی نوار هست. این ورودی میتواند لیست یا رشته باشد که در خط دوم اگر رشته بود هم آن را به لیست تبدیل کردیم. سپس دو لیست تعریف شده به نام های `left_hand_side` و `right_hand_side` که دو سر نوار را نشان میدهند. سپس تعریف کردیم اگر ورودی داشتیم متغیر `current_symbol` برابر اولین کاراکتر رشته ورودی باشد و نوار سمت راستش باقی کاراکتر های ورودی باشد. اگر هم ورودی خالی بود متغیر `current_symbol` با `blank_symbol` پر میشود و سمت راست و چپ نوار آن لیست های خالی میمانند. حال متغیر `state` را تعریف میکنیم که استیتی که در آن هستیم را نشان میدهد و مقدار اولیه آن همان اینیشیال استیت خواهد بود. حال وارد یک حلقه میشویم که در آن `configuration` که شامل استیت کنونی و نوار است در همان ابتدا آپدیت میشود و سپس چک میشود که در چه استیتی هستیم. اگر در `accept_state` یا `reject_state` بودیم پس از `yield` کردن خروجی از حلقه خارج میشویم و در غیر این حالات `(None, configuration)` را `yield` میکنیم. حال در متغیر `to_exe` حالات بعدی که شامل استیت بعدی و جهت حرکت در نوار و رایت کردن یک سیمبول در این خانه میشود را با جست و جو در دیکشنری `transitions` ذخیره میکنیم. در خطوط بعد متغیر های استیت و متغیر های مربوط به نوار را آپدیت میکنیم تا در اجرای دوباره لوپ `configuration` آپدیت شود. در حرکت به سمت چپ چک میشود که از مرز چپ نوار عبور نکنیم چون در حال حاضر

نوار ما از سمت چپ متناهی هست و اگر برنامه بخواهد از آن عبور کند خطا می‌دهیم و به استیت qr که استیت ریجکت هست می‌رویم تا ریجکت کنیم

حالا متد های accepts و rejects را مینویسیم:

```
1 def accepts(self, input_, step_limit=100):
2     for action, _ in islice(self.run(input_), step_limit):
3         if action == 'Accept':
4             return True
5         elif action == 'Reject':
6             return False
7         logging.warning("Step limit reached without halting")
8     return None
9
10 def rejects(self, input_, **kwargs):
11     result = self.accepts(input_, **kwargs)
12     if result is None:
13         return None
14     return not self.accepts(input_, **kwargs)
```

وظیفه این دو متد این است که با دریافت ورودی چک کنند که ماشین تورینگ ما این ورودی را قبول می‌کند یا نه.

حال متد debug را مینویسیم

```
1 def debug(self, input_, step_limit=100, colored=False):
2     for i, (action, configuration) in
3         ↪ enumerate(islice(self.run(input_), step_limit)):
4         state = configuration['state']
5         left = configuration['left_hand_side']
6         symbol = configuration['symbol']
7         right = configuration['right_hand_side']
8         left_part = ''.join(reversed(left))
9         right_part = ''.join(right)
10
11         if colored:
12             tape_str =
13             ↪ f"{left_part}\033[91m[{symbol}]\033[0m{right_part}"
14         else:
15             tape_str = f"{left_part}[{symbol}]{right_part}"
16
17         print(f"Step {i}: state={state}, tape={tape_str}")
18
19         if action in ('Accept', 'Reject'):
20             print(f"Finished with {action}")
21             break
```

در این متد مرحله به مرحله ماشین تورینگ طبق ورودی مشخص اجرا می‌شود و configuration آن چاپ می‌شود تا دیدگاه راحت تری داشته باشیم

```

PS C:\Users\user\Desktop\Folders\IUST\TLA\Project\To-Present> py '.\Part 1\I\student_test_turing.py'
✅ Successfully imported TuringMachine from turing_machine.py!
=====
TURING MACHINE SIMULATOR SELF-CHECK
=====

--- Test 1: Basic Acceptance of a Single Symbol ---
✅ Correctly accepted '#'
✅ Correctly rejected '##'
✅ Correctly rejected '1'
🔥 Test 1 passed successfully!

--- Test 2: Generator and Configuration Output Structure ---
✅ Generator yielded correctly structured tuples.
✅ Configuration dictionary matches the unified interface specification.
🔥 Test 2 passed successfully!

--- Test 3: Tape Boundary Behavior ---
Testing execution that triggers a leftward move from the first cell:
WARNING:root:Crossed left boundary of singly-infinite tape
Step 0: state=q0, tape=[1]
WARNING:root:Crossed left boundary of singly-infinite tape
Step 1: state=qr, tape=[1]1
Finished with Reject
✅ Test 3 ran without crashing.
🔥 Test 3 completed successfully!

=====
🔥 CONGRATULATIONS! Your TuringMachine simulator passes all basic self-tests.
Ready for Part II and Part III! Next, try running the provided examples.
=====

```

شکل ۱: تست خود ارزیابی ماشین تورینگ

۲.۱.۱ فهرست کردن state ها در اسکریپت های تست

برای این منظور به کافی است به ازای هر کدام از اسکریپت ها این تکه کد را ن شود

```

1 def print_states(transition_mapping):
2     states = set()
3     for (start, finish) in transition_mapping.items():
4         (s1, _) = start
5         (s2, _, _) = finish
6         states.add(s1)
7         states.add(s2)
8     print("The Turing machine has", len(states), "states:")
9     for i in states:
10        print(i)
11    print()
12
13 if __name__ == "__main__":
14    print_states(transitions)

```

و این هم نتایج برای هر کدام از استیت ها:

```
PS C:\Users\user\Desktop\Folders\IUST\TLA\Project\To-Present> py '.\Part 1\I\test_turing_machine_example1.py'
The Turing machine has 4 states:
q0
qa
saw_#
saw_##

PS C:\Users\user\Desktop\Folders\IUST\TLA\Project\To-Present> py '.\Part 1\I\test_turing_machine_example2.py'
The Turing machine has 9 states:
Check1
FindDelimiter0
q0
FindLeftmost
FindNext
qa
FindDelimiter1
End
Check0
```

شکل ۲: لیست استیت های اسکریپت ماشین های تورینگ نمونه

۳.۱.۱ نمایش configuration ماشین به ازای هر انتقال

به این منظور نیاز به تعریف همچنین تابعی در هر کدام از اسکریپت ها داریم:

```
1 def run(input_):
2     w = input_
3     print("Input:",w)
4     print("Accepted" if machine.accepts(w) else "Rejected")
5     machine.debug(w, colored=True)
6     print()
```

و این هم نمایش آنها:

```
Input: ##
Accepted
Step 0: state=q0, tape=[#]#
Step 1: state=saw_#, tape=#[#]
Step 2: state=saw_##, tape=##[ ]
Step 3: state=qa, tape=##[ ]
Finished with Accept
```

شکل ۳: نمایش پیکربندی ماشین در اسکریپت اول

```

Input: @#
Accepted
Step 0: state=q0, tape=[@]#
Step 1: state=q0, tape=@[#]
Step 2: state=End, tape=@#[]
Step 3: state=qa, tape=@#[]
Finished with Accept

Input: @0000#XXXX1
Rejected
Step 0: state=q0, tape=[@]0000#XXXX1
Step 1: state=q0, tape=@[0]000#XXXX1
Step 2: state=FindDelimiter0, tape=@x[0]00#XXXX1
Step 3: state=FindDelimiter0, tape=@x0[0]0#XXXX1
Step 4: state=FindDelimiter0, tape=@x00[0]#XXXX1
Step 5: state=FindDelimiter0, tape=@x000[#]XXXX1
Step 6: state=Check0, tape=@x000#[X]XXX1
Step 7: state=Check0, tape=@x000#X[X]XX1
Step 8: state=Check0, tape=@x000#XX[X]X1
Step 9: state=Check0, tape=@x000#XXX[X]1
Step 10: state=Check0, tape=@x000#XXXX[1]
Step 11: state=qr, tape=@x000#XXXX[1]
Finished with Reject

Input: @0111#XXXX1
Rejected
Step 0: state=q0, tape=[@]0111#XXXX1
Step 1: state=q0, tape=@[0]111#XXXX1
Step 2: state=FindDelimiter0, tape=@x[1]11#XXXX1
Step 3: state=FindDelimiter0, tape=@x1[1]1#XXXX1
Step 4: state=FindDelimiter0, tape=@x11[1]#XXXX1
Step 5: state=FindDelimiter0, tape=@x111[#]XXXX1
Step 6: state=Check0, tape=@x111#[X]XXX1
Step 7: state=Check0, tape=@x111#X[X]XX1
Step 8: state=Check0, tape=@x111#XX[X]X1
Step 9: state=Check0, tape=@x111#XXX[X]1
Step 10: state=Check0, tape=@x111#XXXX[1]
Step 11: state=qr, tape=@x111#XXXX[1]
Finished with Reject

```

شکل ۴: نمایش پیکربندی ماشین در اسکریپت دوم

۴.۱.۱ توصیف عملکرد ماشین های تورینگ نمونه

ماشین اول تنها ورودی ## را میپذیرد و هر ورودی دیگری را رد میکند.

```

۱ transitions = {
۲     ('q0', '#'): ('saw_#', '#', 'R'),
۳     ('saw_#', '#'): ('saw_##', '#', 'R'),
۴     ('saw_##', ''): ('qa', '', 'R'),
۵ }

```

ترنزیشن اول به این معناست که رشته باید با # شروع شود و در غیر این صورت ریجکت میکند. اگر این اتفاق افتاد هد نوار به راست میرود و به استیت saw_# میرود که در این استیت هم حتما باید # دیده شود و در این صورت هد نوار به راست میرود و به استیت saw_## میرویم که طبق ترنزیشن آخر

ماشین دوم وظیفه دارد بررسی کند که آیا دو رشته‌ی دودویی قرار گرفته در دو طرف جداکننده‌ی # با یکدیگر یکسان هستند یا خیر. در صورتی که دو رشته کاملاً برابر باشند، ماشین آن‌ها را به تدریج با نماد X جایگزین کرده و در نهایت ورودی را می‌پذیرد؛ در غیر این صورت در یکی از استیت‌ها متوقف شده و رشته را رد می‌کند.

```

1 transitions = {
2     ('q0', '@'): ('q0', '@', 'R'),
3     ('q0', '#'): ('End', '#', 'R'),
4     ('End', ''): ('qa', '', 'R'),
5
6     ('q0', '0'): ('FindDelimiter0', 'X', 'R'),
7     ('FindDelimiter0', '#'): ('Check0', '#', 'R'),
8     ('Check0', '0'): ('FindLeftmost', 'X', 'L'),
9
10    ('q0', '1'): ('FindDelimiter1', 'X', 'R'),
11    ('FindDelimiter1', '#'): ('Check1', '#', 'R'),
12    ('Check1', '1'): ('FindLeftmost', 'X', 'L'),
13
14    ('FindLeftmost', '0'): ('FindLeftmost', '0', 'L'),
15    ('FindLeftmost', '1'): ('FindLeftmost', '1', 'L'),
16    ('FindLeftmost', 'X'): ('FindLeftmost', 'X', 'L'),
17    ('FindLeftmost', '#'): ('FindLeftmost', '#', 'L'),
18    ('FindLeftmost', '@'): ('FindNext', '@', 'R'),
19
20    ('FindNext', 'X'): ('FindNext', 'X', 'R'),
21    ('FindNext', '0'): ('FindDelimiter0', 'X', 'R'),
22    ('FindNext', '1'): ('FindDelimiter1', 'X', 'R'),
23    ('FindNext', '#'): ('End', '#', 'R'),
24
25    ('FindDelimiter0', '0'): ('FindDelimiter0', '0', 'R'),
26    ('FindDelimiter0', '1'): ('FindDelimiter0', '1', 'R'),
27    ('FindDelimiter1', '0'): ('FindDelimiter1', '0', 'R'),
28    ('FindDelimiter1', '1'): ('FindDelimiter1', '1', 'R'),
29
30    ('Check0', 'X'): ('Check0', 'X', 'R'),
31    ('Check1', 'X'): ('Check1', 'X', 'R'),
32
33    ('End', 'X'): ('End', 'X', 'R')
34 }
```

در ابتدای کار اگر اولین کاراکتر ورودی @ باشد، ماشین تنها از روی آن عبور می‌کند. دلیل وجود این کاراکتر این است که اول نوار مشخص باشد تا بتوانیم سمت چپ ترین کاراکتر را در این نوار نیمه متناهی پیدا کنیم. همچنین اگر در همان ابتدای کار جداکننده‌ی # مشاهده شود، به این معناست که هر دو رشته خالی هستند؛ بنابراین ماشین وارد استیت End شده و در نهایت ورودی را می‌پذیرد.

اگر اولین کاراکتر رشته 0 باشد، ماشین آن را با X علامت‌گذاری کرده و وارد استیت FindDelimiter0 می‌شود. به همین ترتیب اگر اولین کاراکتر 1 باشد، وارد استیت FindDelimiter1 خواهد شد. وظیفه‌ی این دو استیت حرکت به سمت راست تا رسیدن به جداکننده‌ی # است. پس از رسیدن به جداکننده، ماشین وارد استیت Check0 یا Check1 می‌شود و اولین

کاراکتر بررسی نشده‌ی سمت راست را با کاراکتری که از سمت چپ انتخاب شده است مقایسه می‌کند. اگر دو کاراکتر برابر باشند، آن کاراکتر نیز با X جایگزین شده و ماشین وارد استیت FindLeftmost می‌شود؛ در غیر این صورت هیچ ترنزیشن معتبری وجود نخواهد داشت و ماشین رشته را رد می‌کند.

در استیت FindLeftmost هد نوار تا ابتدای نوار به سمت چپ حرکت می‌کند و پس از رسیدن به نماد @ وارد استیت FindNext می‌شود. در این استیت تمام کاراکترهای علامت‌گذاری‌شده‌ی X نادیده گرفته می‌شوند تا اولین کاراکتر بررسی نشده پیدا شود.

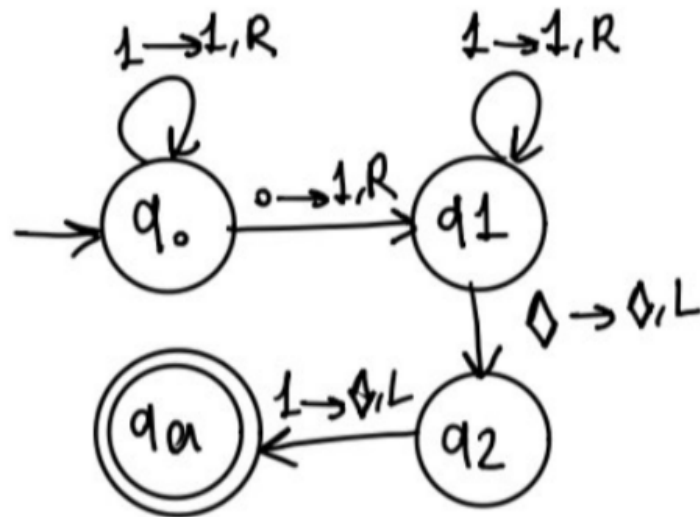
اگر کاراکتر پیدا شده 0 باشد، ماشین دوباره وارد استیت FindDelimiter0 می‌شود و اگر 1 باشد وارد استیت FindDelimiter1 خواهد شد. سپس همین فرآیند برای جفت بعدی کاراکترها تکرار می‌شود.

در نهایت زمانی که استیت FindNext به جداکننده‌ی # برسد، به این معناست که تمام کاراکترهای سمت چپ بررسی و با X جایگزین شده‌اند. در این حالت ماشین وارد استیت End می‌شود و اگر پس از عبور از تمام Xهای سمت راست به نماد خانه‌ی خالی برسد، وارد استیت نهایی qa شده و ورودی را می‌پذیرد.

۲.۱ توابع در ماشین تورینگ

۱.۲.۱ جمع کننده‌ی یک یکی

در این تابع ورودی دو عدد یک یکی هستند که با ۰ از همدیگه جدا شده اند و خروجی باید حاصل جمع این دو عدد بشود. تصویر ماشین طراحی شده:



شکل ۵: ماشین جمع کننده‌ی یک یکی

ترنزیشن های ماشین بالا به این صورت در کد خواهند بود:

```

۱ transitions = {
۲     ('q0', '1'): ('q0', '1', 'R'),
۳     ('q0', '0'): ('q1', '1', 'R'),
۴     ('q1', '1'): ('q1', '1', 'R'),
۵     ('q1', ''): ('q2', '', 'L'),
۶     ('q2', '1'): ('qa', '', 'L'),
۷ }

```

منطق این ماشین اینگونه است که صفر جدا کننده بین اعداد را پاک میکند و این دو عدد را به هم میچسباند.

تصویر خروجی این ماشین به ازای ورودی ۱۱۰۱۱ را نمایش میدهم:

```

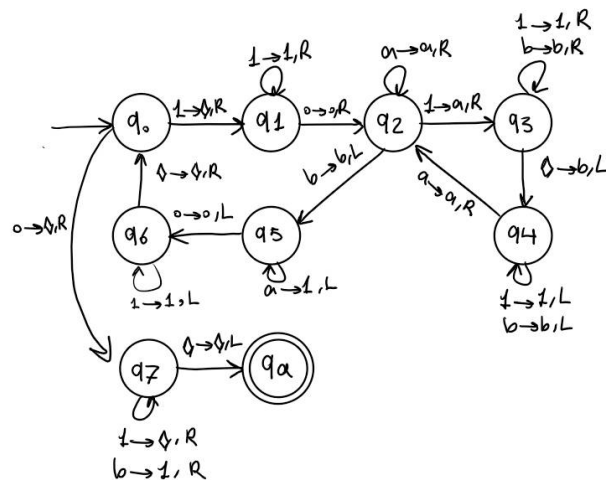
Input: 110111
Accepted
Step 0: state=q0, tape=[1]10111
Step 1: state=q0, tape=1[1]0111
Step 2: state=q0, tape=11[0]111
Step 3: state=q1, tape=111[1]11
Step 4: state=q1, tape=1111[1]1
Step 5: state=q1, tape=11111[1]
Step 6: state=q1, tape=111111[ ]
Step 7: state=q2, tape=11111[1]
Step 8: state=qa, tape=1111[1]
Finished with Accept

```

شکل ۶: خروجی جمع کننده‌ی یک یکی به ازای ورودی ۱۱۰۱۱۱

۲.۲.۱ ضرب کننده‌ی یک یکی

این ماشین هم وظیفه دارد دو عدد یک یکی که با صفر از همدیگه جدا شدند را در هم ضرب کند.
تصویر ماشین طراحی شده:



شکل ۷: ماشین ضرب کننده‌ی یک یکی

ترنزیشن های ماشین بالا به این صورت خواهند بود:

```

۱ transitions = {
۲     ('q0', '1'): ('q1', '', 'R'),
۳     ('q0', '0'): ('q7', '', 'R'),
۴     ('q1', '1'): ('q1', '1', 'R'),
۵     ('q1', '0'): ('q2', '0', 'R'),
۶     ('q2', 'a'): ('q2', 'a', 'R'),
۷     ('q2', 'b'): ('q5', 'b', 'L'),
۸     ('q2', '1'): ('q3', 'a', 'R'),
۹     ('q3', '1'): ('q3', '1', 'R'),
۱۰    ('q3', 'b'): ('q3', 'b', 'R'),
۱۱    ('q3', '') : ('q4', 'b', 'L'),
۱۲    ('q4', '1'): ('q4', '1', 'L'),
۱۳    ('q4', 'b'): ('q4', 'b', 'L'),
۱۴    ('q4', 'a'): ('q2', 'a', 'R'),
۱۵    ('q5', 'a'): ('q5', '1', 'L'),
۱۶    ('q5', '0'): ('q6', '0', 'L'),
۱۷    ('q6', '1'): ('q6', '1', 'L'),
۱۸    ('q6', '') : ('q0', '', 'R'),
۱۹    ('q7', '1'): ('q7', '', 'R'),
۲۰    ('q7', 'b'): ('q7', '1', 'R'),
۲۱    ('q7', '') : ('qa', '', 'L'),
۲۲ }

```

منطق این ماشین اینگونه است که به ازای هر یک در عدد اول، به تعداد یک های عدد دوم به جواب ۱ اضافه میکند تا در نهایت به اندازه ضرب دو عدد یک داشته باشیم. تصویر خروجی این ماشین به ازای ورودی ۰۱۱۱۱ را نمایش میدهیم:

```

Input: 01111
Accepted
Step 0: state=q0, tape=[0]1111
Step 1: state=q7, tape=[1]111
Step 2: state=q7, tape=[1]11
Step 3: state=q7, tape=[1]1
Step 4: state=q7, tape=[1]
Step 5: state=q7, tape=[]
Step 6: state=qa, tape=[]
Finished with Accept

```

شکل ۸: خروجی ضرب کننده‌ی یک یکی به ازای ورودی ۰۱۱۱۱

۳.۱ مسئله بیزی بیور

۱.۳.۱ چرا پیدا کردن برنامه های بیدستر پرتلاش کار ساده ای نیست و به عنوان یک مسئله غیر قابل حل شناخته میشود؟

مسئله‌ی یافتن برنامه‌ی بیزی بیور به این دلیل یک مسئله‌ی غیرقابل حل محسوب می‌شود که برای تعیین مقدار آن باید بتوان تشخیص داد آیا یک ماشین تورینگ پس از شروع اجرا متوقف می‌شود یا برای همیشه به اجرای خود ادامه می‌دهد. ماشین بیزی بیور به گونه‌ای تعریف می‌شود که در میان تمام ماشین‌های تورینگ با تعداد مشخصی حالت، بیشترین تعداد گام را قبل از توقف اجرا کند تا بیشترین تعداد نماد را روی نوار بنویسد. برای پیدا کردن چنین ماشینی لازم است تمام ماشین‌های ممکن بررسی شوند. اما در هنگام بررسی، مشخص نیست که یک ماشین واقعاً هیچ‌گاه متوقف نخواهد شد یا فقط پس از تعداد بسیار زیادی گام متوقف می‌شود. این موضوع مستقیماً به مسئله‌ی توقف مرتبط است. آلن تورینگ ثابت کرد که هیچ الگوریتم عمومی وجود ندارد که بتواند برای هر ماشین تورینگ و هر ورودی، تشخیص دهد آیا آن ماشین متوقف می‌شود یا خیر. بنابراین اگر الگوریتمی برای یافتن دقیق ماشین بیزی بیور وجود داشت، می‌توانست از آن برای حل مسئله‌ی توقف نیز استفاده شود؛ در حالی که می‌دانیم مسئله‌ی توقف غیرقابل حل است. در نتیجه، مسئله‌ی یافتن برنامه‌ی بیزی بیور نیز غیرقابل حل خواهد بود. به همین دلیل، مقادیر دقیق تابع بیزی بیور تنها برای ماشین‌هایی با تعداد حالت کم محاسبه شده‌اند و با افزایش تعداد حالت‌ها، تعداد ماشین‌های ممکن به صورت انفجاری رشد می‌کند. علاوه بر این، تشخیص اینکه کدام ماشین در نهایت متوقف می‌شود نیز امکان‌پذیر نیست.

۲.۳.۱ تبدیل نوار ماشین تورینگ از یک طرف متناهی به نامتناهی

برای این منظور تنها کافیست منطق حرکت به سمت چپ در متد run در کلاس ماشین تورینگ خود را تغییر دهیم.

```
1 elif to_exe[2] == 'L':
2     right_hand_side.insert(0, to_exe[1])
3     if left_hand_side:
4         current_symbol = left_hand_side.pop(0)
5     else:
6         current_symbol = self.blank_symbol
```

به این ترتیب اگر خواستیم در نوار به سمت چپ حرکت کنیم و سمت چپ خانه فعلی نوار ما خالی بود current_symbol را برابر blank symbol قرار می‌دهیم. نوار سمت راست هم که از قبل آپدیت شده و نوار سمت چپ خالی میماند.

۳.۳.۱ چرا استفاده از ژنراتورهای پایتون در شبیه سازی ماشین های تورینگ سودمند است؟

استفاده از ژنراتورهای پایتون در شبیه‌سازی ماشین تورینگ باعث می‌شود اجرای ماشین به صورت مرحله‌به‌مرحله انجام شود. در این پروژه، متد run به جای اینکه کل محاسبه را یکجا اجرا کند، پس از هر انتقال با استفاده از دستور yield وضعیت فعلی ماشین شامل استیت، محتوای نوار و محل هد را برمی‌گرداند و اجرای برنامه را تا درخواست مرحله‌ی بعدی متوقف می‌کند.

این روش چند مزیت مهم دارد. نخست اینکه امکان مشاهده و بررسی روند اجرای ماشین را فراهم می‌کند؛ به طوری که می‌توان در هر مرحله وضعیت ماشین را نمایش داد یا برای اشکال‌زدایی از آن استفاده کرد. به همین دلیل در این پروژه متد debug تنها با پیمایش خروجی ژنراتور، تمام مراحل اجرای ماشین را چاپ می‌کند و نیازی به تغییر منطق اصلی شبیه‌سازی ندارد.

مزیت دیگر ژنراتورها، بهینه بودن مصرف حافظه است. اگر تمام وضعیت‌های ماشین در یک لیست ذخیره می‌شدند، برای ماشین‌هایی که تعداد گام‌های زیادی دارند حافظه‌ی زیادی مصرف می‌شد تا گام‌ها را ذخیره کنیم و بعداً چاپ کنیم. اما ژنراتور در هر لحظه فقط وضعیت جاری را تولید می‌کند و پس از پردازش آن، به مرحله‌ی بعد می‌رود؛ بنابراین حافظه‌ی مصرفی تقریباً ثابت باقی می‌ماند.

همچنین ژنراتورها امکان توقف زودهنگام اجرای ماشین را فراهم می‌کنند. برای مثال در متدهای accepts و rejects تنها تا زمانی که ماشین به حالت پذیرش یا رد برسد، خروجی ژنراتور بررسی می‌شود و در صورت نیاز با استفاده از محدودیت تعداد گام‌ها (step_limit) می‌توان اجرای ماشین را متوقف کرد. این ویژگی به‌ویژه برای ماشین‌هایی که ممکن است وارد حلقه‌ی بی‌نهایت شوند بسیار مفید است زیرا می‌توانیم یک محدودیت گام تعریف کنیم و تا آن گام این ماشین بینهایت را بررسی کنیم.

نکته‌ی مهم دیگر این است که ژنراتورها کنترل اجرای برنامه را در اختیار فراخواننده (Caller) قرار می‌دهند. به عبارت دیگر، متد run تصمیم نمی‌گیرد که ماشین تا انتها اجرا شود؛ بلکه پس از هر انتقال با دستور yield متوقف شده و کنترل را به برنامه‌ی فراخواننده بازمی‌گرداند. سپس هر زمان که فراخواننده با استفاده از next() یا پیمایش ژنراتور درخواست کند، اجرای ماشین دقیقاً از همان نقطه ادامه پیدا می‌کند. این ویژگی باعث می‌شود کاربر بتواند ماشین را تنها به اندازه‌ی نیاز اجرا کند؛ برای مثال تنها یک گام، چند گام یا تا رسیدن به یک شرط خاص.

اجرای تنبل (Lazy Evaluation) به معنی آن است که تا زمانی که به نتیجه‌ی محاسباتی نیازی نداشته‌ایم آن محاسبات را انجام ندهیم. اجرای تنبل ژنراتورها باعث می‌شود شبیه‌سازی ماشین‌های تورینگ که ممکن است تعداد بسیار زیادی گام اجرا شوند یا حتی هیچ‌گاه متوقف نشوند نیز امکان‌پذیر باشد. در چنین شرایطی نیازی نیست کل اجرای ماشین از ابتدا تا انتها انجام شود؛ بلکه وضعیت‌ها تنها در زمان درخواست تولید می‌شوند. به همین دلیل، در این پروژه متدهای accepts و rejects تنها تا تعداد مشخصی گام (step_limit) ماشین را اجرا می‌کنند و اگر ماشین در این بازه متوقف نشود، نتیجه‌ی نامشخص (step_limit) برگردانده می‌شود. این روش از گیر کردن برنامه در حلقه‌های بی‌نهایت جلوگیری کرده و امکان کنترل کامل اجرای ماشین را در اختیار کاربر قرار می‌دهد.

در مجموع، استفاده از ژنراتورها باعث شده است شبیه‌سازی ماشین تورینگ به صورت مرحله‌ای، کم‌مصرف، قابل اشکال‌زدایی و مناسب برای مدیریت ماشین‌های با اجرای طولانی یا حتی نامتناهی انجام شود.

۴.۳.۱ اجرای برنامه‌ی دو کارتی بیزی بیور

کارت‌ها به این صورت در فایل busy_beaver تعریف شدند:

```

۱ {
۲     'a0' : ('b', '1', 'r'),
۳     'a1' : ('b', '1', '1'),
۴     'b0' : ('a', '1', '1'),
۵     'b1' : ('h', '1', '1')
۶ }
```

که ترنژیشن های متناظرش در فایل test_busy_beaver_2 به این صورت خواهد بود:

```

۱ bbeaver2 = TuringMachine(
۲     {
۳         ('a', '0') : ('b', '1', 'R'),
۴         ('a', '1') : ('b', '1', 'L'),
۵         ('b', '0') : ('a', '1', 'L'),
۶         ('b', '1') : ('h', '1', 'L')
۷     },
۸     start_state='a', accept_state='h', reject_state='r',
۹     blank_symbol='0'
)

```

نمایش نتیجه:

```

BB with 2 states
Step 0: state=a, tape=[0]00000000000000
Step 1: state=b, tape=1[0]000000000000
Step 2: state=a, tape=[1]10000000000000
Step 3: state=b, tape=[0]11000000000000
Step 4: state=a, tape=[0]1110000000000000
Step 5: state=b, tape=1[1]1100000000000000
Step 6: state=h, tape=[1]1110000000000000
Finished with Accept

```

شکل ۹: نتیجه بیزی بیور ۲ کارته

همانطور که مشاهده میشود ۴ تا عدد ۱ به دست آمده. این نتیجه دقیقاً مطابق با بهترین برنامه ۲ کارته شناخته شده است و برنامه‌ی دو کارتی متوقف شونده‌ی دیگری وجود ندارد که بیشتر از این تعداد ۱ تولید کند.

۵.۳.۱ بیزی بیور ۳ و ۴ کارته

سه کارته:

کارت ها به این صورت هستند که در فایل busy_beaver تعریف شدند:

```

۱ {
۲     'a0' : ('b', '1', 'r'),
۳     'a1' : ('c', '1', '1'),
۴     'b0' : ('a', '1', '1'),
۵     'b1' : ('b', '1', 'r'),

```



```

۶     'c0' : ('b', '1', '1'),
۷     'c1' : ('h', '1', 'r')
۸ }

```

که ترنزیشن های متناظرش در فایل test_busy_beaver_2 به این صورت خواهد بود:

```

۱ bbeaver3 = TuringMachine(
۲     {
۳         ('a', '0'): ('b', '1', 'R'),
۴         ('a', '1'): ('c', '1', 'L'),
۵         ('b', '0'): ('a', '1', 'L'),
۶         ('b', '1'): ('b', '1', 'R'),
۷         ('c', '0'): ('b', '1', 'L'),
۸         ('c', '1'): ('h', '1', 'R')
۹     },
۱۰     start_state='a', accept_state='h', reject_state='r',
    ↪     blank_symbol='0'
۱۱ )

```

نمایش نتیجه:

```

BB with 3 states
Step 0: state=a, tape=[0]000000000000
Step 1: state=b, tape=1[0]000000000000
Step 2: state=a, tape=[1]100000000000
Step 3: state=c, tape=[0]110000000000
Step 4: state=b, tape=[0]111000000000
Step 5: state=a, tape=[0]111100000000
Step 6: state=b, tape=1[1]111000000000
Step 7: state=b, tape=11[1]110000000000
Step 8: state=b, tape=111[1]100000000000
Step 9: state=b, tape=1111[1]000000000000
Step 10: state=b, tape=1111[0]000000000000
Step 11: state=a, tape=1111[1]100000000000
Step 12: state=c, tape=111[1]110000000000
Step 13: state=h, tape=1111[1]100000000000
Finished with Accept

```

شکل ۱۰: نتیجه بیزی بیور ۳ کارته

همانطور که مشاهده میشود موفق شدیم برنامه بیزی بیور ۳ کارته ای بنویسیم که ۶ عدد ۱ در نوار برای ما میسازد.

چهار کارته:

کارت ها به این صورت هستند که در فایل busy_beaver تعریف شدند:

```

1 {
2     'a0': ('b', '1', 'r'),
3     'a1': ('c', '1', 'l'),
4     'b0': ('a', '1', 'l'),
5     'b1': ('b', '1', 'r'),
6     'c0': ('d', '1', 'l'),
7     'c1': ('c', '1', 'r'),
8     'd0': ('a', '1', 'l'),
9     'd1': ('h', '1', 'l')
10 }

```

که ترنزیشن های متناظرش در فایل test_busy_beaver_2 به این صورت خواهد بود:

```

1 bbeaver4 = TuringMachine(
2     {
3         ('a', '0'): ('b', '1', 'R'),
4         ('a', '1'): ('c', '1', 'L'),
5         ('b', '0'): ('a', '1', 'L'),
6         ('b', '1'): ('b', '1', 'R'),
7         ('c', '0'): ('d', '1', 'L'),
8         ('c', '1'): ('c', '1', 'R'),
9         ('d', '0'): ('a', '1', 'L'),
10        ('d', '1'): ('h', '1', 'L')
11    },
12    start_state='a', accept_state='h', reject_state='r',
13    ↪ blank_symbol='0'
14 )

```

نمایش نتیجه:

```

Step 12: state=c, tape=111[1]110000000000
Step 13: state=c, tape=1111[1]100000000000
Step 14: state=c, tape=11111[1]000000000000
Step 15: state=c, tape=111111[0]000000000000
Step 16: state=d, tape=11111[1]100000000000
Step 17: state=h, tape=1111[1]110000000000
Finished with Accept

```

شکل ۱۱: نتیجه بیزی بیور ۴ کارته

که برنامه ما یک ۱ از بیزی بیور ۳ کارته بیشتر تولید کرد که الزاما بهترین برنامه‌ی بیزی بیور ۴ کارته نیست!

۶.۳.۱ بیزی بیور ۵ کارته

کارت ها به این صورت هستند که در فایل busy_beaver تعریف شدند:

```

۱ {
۲     'a0': ('b', '1', 'r'),
۳     'a1': ('b', '1', 'l'),
۴     'b0': ('a', '1', 'l'),
۵     'b1': ('c', '0', 'l'),
۶     'c0': ('e', '1', 'r'),
۷     'c1': ('d', '1', 'l'),
۸     'd0': ('d', '1', 'r'),
۹     'd1': ('a', '0', 'r'),
۱۰    'e0': ('h', '1', 'r'),
۱۱    'e1': ('h', '1', 'r')
۱۲ }

```

که ترنزیشن های متناظرش در فایل test_busy_beaver_2 به این صورت خواهد بود:

```

۱ bbeaver5 = TuringMachine(
۲     {
۳         ('a', '0'): ('b', '1', 'R'),
۴         ('a', '1'): ('b', '1', 'L'),
۵         ('b', '0'): ('a', '1', 'L'),
۶         ('b', '1'): ('c', '0', 'L'),
۷         ('c', '0'): ('e', '1', 'R'),
۸         ('c', '1'): ('d', '1', 'L'),
۹         ('d', '0'): ('d', '1', 'R'),
۱۰        ('d', '1'): ('a', '0', 'R'),
۱۱        ('e', '0'): ('h', '1', 'R'),
۱۲        ('e', '1'): ('h', '1', 'R')
۱۳     },
۱۴     start_state='a', accept_state='h', reject_state='r',
        ↪ blank_symbol='0'
۱۵ )

```

نمایش نتیجه:

```

Step 102: state=a, tape=110[1]11111111110000000000
Step 103: state=b, tape=11[0]11111111110000000000
Step 104: state=a, tape=1[1]11111111110000000000
Step 105: state=b, tape=[1]111111111110000000000
Step 106: state=c, tape=[0]0111111111110000000000
Step 107: state=e, tape=1[0]111111111110000000000
Step 108: state=h, tape=11[1]111111111110000000000
Finished with Accept

```

شکل ۱۲: نتیجه بیزی بیور ۵ کارته

در این بخش یک ماشین تورینگ ۵ کارته طراحی شد که پیش از توقف، ۱۴ نماد ۱ روی نوار تولید می‌کند. با اجرای شبیه‌سازی مشاهده شد که ماشین پس از تعداد مشخصی گام

متوقف شده و وارد حالت نهایی می‌شود؛ بنابراین در مورد این ماشین خاص، متوقف شدن آن به صورت تجربی تأیید شده است.

با این حال، نمی‌توان ادعا کرد که این ماشین بهترین برنامه‌ی ۵ کارته است. برای اثبات چنین ادعایی باید تمام ماشین‌های تورینگ ۵ کارته‌ی ممکن بررسی شوند و مشخص شود که هر یک یا متوقف می‌شوند یا هرگز متوقف نخواهند شد. اما تشخیص توقف یا عدم توقف یک ماشین تورینگ در حالت کلی همان مسئله‌ی توقف (Halting Problem) است که تورینگ ثابت کرده است هیچ الگوریتمی برای حل آن وجود ندارد.

به همین دلیل، حتی اگر ماشینی پیدا کنیم که تعداد زیادی نماد ۱ تولید کند، همواره این احتمال وجود دارد که ماشین ۵ کارته‌ی دیگری وجود داشته باشد که تعداد بیشتری نماد تولید کند یا پس از تعداد بسیار زیادی گام متوقف شود. از آنجا که نمی‌توان برای همه‌ی ماشین‌ها تشخیص داد که آیا در نهایت متوقف می‌شوند یا خیر، اثبات بهینه بودن یک برنامه‌ی جدید کار ساده‌ای نیست.

دلیل اینکه این ماشین را متوقف‌شونده در نظر گرفته‌ایم، این است که در شبیه‌سازی انجام‌شده، ماشین پس از تعداد محدودی گام به حالت توقف رسیده و اجرای آن پایان یافته است. بنابراین برای این ماشین خاص، توقف به صورت تجربی مشاهده شده است؛ هرچند برای یک ماشین ناشناخته نمی‌توان چنین نتیجه‌ای را بدون اجرای آن یا داشتن اثبات اختصاصی به دست آورد. بهترین برنامه بیزی بیور ۵ کارته شناخته شده تا به امروز ۴۰۹۸ تا ۱ تولید میکند و بیش از ۴۱ میلیون گام دارد!

۷.۳.۱ بررسی بهترین بودن برنامه های قبلی

با بررسی مراجع به این نتیجه می‌رسیم که برنامه بیزی بیور ۲ و ۳ کارته ما بهترین برنامه ها هستند و بیشترین تعداد ممکن ۱ را تولید میکنند اما برنامه ۴ کارته ما تنها ۷ عدد ۱ تولید میکرد که متاسفانه بهترین برنامه ممکن نیست و حداکثر عدد یکی که میتوان با ۴ کارت ساخت ۱۳ یک است نه ۷ یک! کارت ها به این صورت هستند که در فایل busy_beaaver تعریف شدند:

```

۱ {
۲     'a0': ('b', '1', 'r'),
۳     'a1': ('b', '1', '1'),
۴     'b0': ('a', '1', '1'),
۵     'b1': ('c', '0', '1'),
۶     'c0': ('h', '1', 'r'),
۷     'c1': ('d', '1', '1'),
۸     'd0': ('d', '1', 'r'),
۹     'd1': ('a', '0', 'r')
۱۰ }
```

که ترنزیشن های متناظرش در فایل test_busy_beaaver_2 به این صورت خواهد بود:

```

۱ bbeaver5 = TuringMachine(
۲     {
۳         ('a', '0'): ('b', '1', 'R'),
۴         ('a', '1'): ('b', '1', 'L'),
۵         ('b', '0'): ('a', '1', 'L'),
۶         ('b', '1'): ('c', '0', 'L'),
۷         ('c', '0'): ('h', '1', 'R'),
۸         ('c', '1'): ('d', '1', 'L'),
```

```

۹      ('d', '0'): ('d', '1', 'R'),
۱۰      ('d', '1'): ('a', '0', 'R')
۱۱  },
۱۲  start_state='a', accept_state='h', reject_state='r',
    ↪ blank_symbol='0'
۱۳  )

```

نمایش نتیجه:

```

Step 104: state=a, tape=1[1]111111111110000000000
Step 105: state=b, tape=[1]1111111111110000000000
Step 106: state=c, tape=[0]01111111111110000000000
Step 107: state=h, tape=1[0]1111111111110000000000
Finished with Accept

```

شکل ۱۳: نتیجه بهترین بیزی بیور ۴ کارته

همانطور که مشاهده میشود این برنامه در ۱۰۷ گام ۱۳ یک تولید میکند.

۲ اتوماتا های سلولی

۱.۲ بازی زندگی کانوی

۱.۱.۲ پیاده سازی قوانین چهار گانه‌ی بازی زندگی

این قسمت از بازی زندگی در متد evolve در فایل conway.py پیاده سازی میشود.

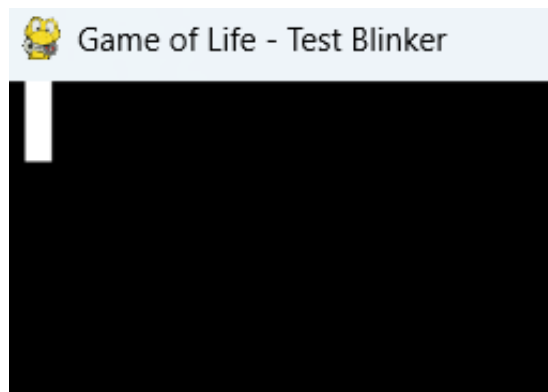
```
1 def evolve(self):
2     if self.fastMode:
3         self.grid = self.update_grid_fast(self.grid)
4     else:
5         next_grid = np.zeros((self.rows, self.cols), np.uint)
6         for i in range(self.rows):
7             for j in range(self.cols):
8                 num_of_allive_neighbors = 0
9                 if self.finite:
10                     if i-1 < 0:
11                         previous_row = i
12                     else:
13                         previous_row = i - 1
14                     if i+1 == self.rows:
15                         next_row = i
16                     else:
17                         next_row = i + 1
18                     if j-1 < 0:
19                         previous_col = j
20                     else:
21                         previous_col = j - 1
22                     if j+1 == self.cols:
23                         next_col = j
24                     else:
25                         next_col = j + 1
26                 else:
27                     previous_row = (i - 1) % self.rows
28                     next_row = (i + 1) % self.rows
29                     previous_col = (j - 1) % self.cols
30                     next_col = (j + 1) % self.cols
31
32                 if (not self.finite) and (previous_col > next_col or
33 ↪ previous_row > next_row):
34                     for count_row in (previous_row, i, next_row):
35                         for count_column in (previous_col, j,
36 ↪ next_col):
37                             if count_row == i and count_column == j:
38                                 continue
39                             num_of_allive_neighbors +=
40 ↪ self.grid[count_row, count_column]
41
42                 else:
43                     for count_row in range(previous_row, next_row + 1):
```

```

۴۰         for count_column in range(previous_col,
۴۱             ↪ next_col + 1):
۴۲             if count_row == i and count_column == j:
۴۳                 continue
۴۴                 num_of_allive_neighbors +=
۴۵                 ↪ self.grid[count_row, count_column]
۴۶
۴۷         if self.grid[i,j] == 1 and num_of_allive_neighbors < 2:
۴۸             next_grid[i,j] = 0
۴۹         elif self.grid[i,j] == 1 and (num_of_allive_neighbors
۵۰             ↪ == 2 or num_of_allive_neighbors == 3):
۵۱             next_grid[i,j] = 1
۵۲         elif self.grid[i,j] == 1 and num_of_allive_neighbors >
۵۳             ↪ 3:
۵۴             next_grid[i,j] = 0
۵۵         elif self.grid[i,j] == 0 and num_of_allive_neighbors ==
۵۶             ↪ 3:
۵۷             next_grid[i,j] = 1
۵۸         self.grid = next_grid

```

در این متد ابتدا چک میشود که آیا فست مود فعال است یا خیر. اگر فعال بود متد `update_grid_fast` اجرا میشود که در ادامه توضیح داده خواهد شد. حال که فست مود غیر فعال است، ابتدا یک `grid` جدید تعریف میکنیم که تمام سلول هایش مرده هستند. این کار برای این انجام میشود چون اگر بخواهیم به صورت داینامیک سلول ها را آپدیت کنیم بر وضعیت سلول های همسایه آنها تاثیر میگذارد برای همین مهم است حتما تغییرات را در یک `grid` جدید ساخت و سپس آن را مقدار دهی کرد. حال با یک حلقه‌ی تو در تو تک به تک سلول ها را بررسی میکنیم تا حالت بعدی آنها را مشخص کنیم. متغیر `num_of_allive_neighbors` به این منظور تعریف میشود که تعداد همسایه های زنده‌ی سلول در حال بررسی را ذخیره کنیم تا دربارهی حالت بعدی آن تصمیم بگیریم. حالا متناسب با فعال بودن `finite` تعداد همسایه های زنده‌ی این سلول را محاسبه میکنیم. اگر فعال بود یعنی `wrap` نمیکند و سلول های مرزی همسایه های بیرون از `grid` مرده حساب میشوند. در غیر این صورت آن مرز های `grid` به هم دیگر وصل میشوند و `grid` مانند یک کره فرض میشود. متناسب با این شرایط در متغیر های مختلف شماره اندیس ردیف و ستون قبلی و بعدی این سلول فعلی را ذخیره میکنیم. حال که اندیس های ردیف و ستون های قبلی و بعدی را داریم به راحتی میتوانیم تعداد همسایه ها را حساب کنیم. در خطوط بعدی با ۴ شرط `if` ساده قوانین `gol` متناسب با حالت فعلی سلول و تعداد همسایه های زنده‌اش مشخص میشود و در خط آخر این `grid` مقدار دهی میشود.



شکل ۱۴: تست متد evolve

۲.۱.۲ گلايدر و اهميت آن در GOL

ابتدا متد `insert_glider` را در پايگيم اجرا ميکنيم تا گلايدر را مشاهده کنيم.



شکل ۱۵: تصوير گلايدر

در بازی زندگی کانوی، گلايدر (Glider) یکی از ساده‌ترین و مهم‌ترین الگوهای متحرک است. گلايدر از پنج سلول زنده تشکیل شده است و پس از هر چهار نسل، دوباره همان شکل اولیه‌ی خود را به دست می‌آورد، با این تفاوت که یک خانه به صورت مورب جابه‌جا شده است. به همین دلیل سرعت حرکت آن برابر با $c/4$ است؛ یعنی در هر چهار نسل، به اندازه‌ی یک خانه در راستای قطر حرکت می‌کند.

اهمیت گلايدر تنها به متحرک بودن آن محدود نمی‌شود. از آنجا که گلايدر می‌تواند بدون از بین رفتن در صفحه حرکت کند، از آن به عنوان حامل اطلاعات استفاده می‌شود. در بسیاری از مدارهای منطقی ساخته‌شده در بازی زندگی، وجود یا عدم وجود یک گلايدر در زمان مشخص، به ترتیب بیانگر مقدار منطقی ۱ و ۰ است. بنابراین گلايدها نقش سیگنال‌هایی را ایفا می‌کنند

که اطلاعات را از بخشی از مدار به بخش دیگر منتقل می‌کنند. یکی دیگر از ویژگی‌های مهم گلايدر، قابلیت تعامل آن با سایر ساختارهای بازی زندگی است. برخورد گلايدر با گلايدهای دیگر یا با الگوهای ثابت می‌تواند باعث نابودی، تغییر مسیر، تولید گلايدهای جدید یا ایجاد ساختارهای دیگر شود. با استفاده از همین برخوردهای کنترل‌شده می‌توان گیت‌های منطقی مانند AND، OR و NOT را پیاده‌سازی کرد و حتی مدارهای محاسباتی پیچیده‌تری ساخت.

در این پروژه نیز از گلايدر به عنوان واحد انتقال اطلاعات استفاده شده است. با تعیین محل، جهت و زمان تولید گلايدها، امکان ارسال سیگنال میان بخش‌های مختلف صفحه فراهم می‌شود و در نهایت با استفاده از برخورد آن‌ها، عملیات منطقی مورد نظر انجام می‌گیرد. به همین دلیل، گلايدر یکی از بنیادی‌ترین اجزای محاسبات در بازی زندگی کانوی به شمار می‌رود.

۳.۱.۲ پیاده سازی پارسر فایل های الگو

این قسمت در تابع `parse_pattern` پیاده سازی میشود.

```

1 def parse_pattern(filepath):
2     FileFormat = os.path.splitext(filepath)[1].lower()
3     if FileFormat == ".cells":
4         FileLines = []
5         with open(filepath, "r") as File:
6             for line in File:
7                 if line.startswith("!"):
8                     continue
9                 if not line:
10                    continue
11                line.rstrip("\n")
12                FileLines.append(line)
13            height = len(FileLines)
14            width = 0
15            if FileLines:
16                width = max(len(line) for line in FileLines)
17            result = []
18            for i in range(height):
19                for j in range(len(FileLines[i])):
20                    if FileLines[i][j] == ".":
21                        continue
22                    if FileLines[i][j] == "0":
23                        result.append((i, j))
24            return width, height, result
25    elif FileFormat == ".rle":
26        row = 0
27        col = 0
28        width = 0
29        height = 0
30        result = []
31        with open(filepath, "r") as File:
32            isEnd = False
33            for line in File:
34                if isEnd:

```

```

۳۵         break
۳۶     if line.startswith("#"):
۳۷         continue
۳۸     line = line.strip()
۳۹     if not line:
۴۰         continue
۴۱     if line.startswith("x"):
۴۲         match =
→         re.match(r"x\s*=\s*(\d+)\s*,\s*y\s*=\s*(\d+)",
→         line)
۴۳     if match:
۴۴         width = int(match.group(1))
۴۵         height = int(match.group(2))
۴۶         continue
۴۷     number = ""
۴۸     for ch in line:
۴۹         if ch.isdigit():
۵۰             number += ch
۵۱             continue
۵۲         else:
۵۳             if number:
۵۴                 repeat = int(number)
۵۵             else:
۵۶                 repeat = 1
۵۷             number = ""
۵۸             if ch == "b":
۵۹                 col += repeat
۶۰                 continue
۶۱             elif ch == "$":
۶۲                 row += repeat
۶۳                 col = 0
۶۴                 continue
۶۵             elif ch in "xyz":
۶۶                 for _ in range(repeat):
۶۷                     result.append((row,col))
۶۸                     col += 1
۶۹             elif ch == "!":
۷۰                 isEnd = True
۷۱                 break
۷۲     return width, height, result
۷۳ else:
۷۴     raise ValueError(f"Unsupported file format: {FileFormat}")

```

این فانکشن یک ورودی دارد که آدرس فایلی است که قرار است پارس شود. این فایل میتواند از نوع (.cells) یا (.rle) باشد. پس باید ابتدا فرمتش را به دست آورد و در متغیری مثل FileFormat ذخیره کرد.

برای فرمت cells، خطوطی که در ابتدای آن ها ! باشد به عنوان کامنت هستند و باید نادیده گرفته شوند. در خطوط دیگر نقطه (.) به معنی سلول مرده و حرف بزرگ او (0) به معنی سلول زنده خواهند بود. پس یک حلقه بین تمام خطوط این فایل میزنیم و خطوط خالی یا کامنت را نادیده میگیریم و سپس آن خط را به عنوان یک عضو به لیست FileLines اضافه میکنیم.

ارتفاع `grid` ما میشود تعداد اعضای لیست `FileLines` و عرض آن میشود طول بزرگترین عضو این لیست. حال یک حلقه تو در تو در این لیست میزنیم و مختصات سلول های زنده را ذخیره میکنیم و در نهایت ارتفاع و عرض و مختصات سلول های زنده را برمیگردانیم. برای فرمت `rle`، خطوطی که اول آنها `#` باشد کامنت هستند. خط اول به فرم `x = m, y = n` است که در آن `m` نشان دهنده عرض و `y` نشان دهنده طول میباشد. در خطوط پس از آن حرف `b` نشان دهنده سلول مرده و حرف `o` نشان دهنده سلول زنده و علامت `$` به معنای انتهای خط است. همچنین اگر عددی قبل هر یک از این کاراکتر ها بیاید به این معناست که به آن تعداد این کاراکتر ایفای نقش میکند. همچنین علامت تعجب (!) به معنی انتهای فایل میباشد. پس در پارسر یک حلقه بین خطوط فایل میزنیم و هر خطی که با `#` شروع میشود را نادیده میگیریم. خطی که با `x` شروع میشود همان خطی است که حاوی اطلاعات درمورد عرض و ارتفاع `grid` است. این دو مقدار را استخراج و مقداردهی میکنیم. در خطوط دیگر با آپدیت کردن شماره اندیس ردیف و ستون مختصات سلول های زنده را پیدا و به لیست اضافه میکنیم و در انتها عرض و ارتفاع و مختصات سلول های زنده را برمیگردانیم. ابتدا متد `insert_glider` را در پایگیم اجرا میکنیم تا گلايدر را مشاهده کنیم.



شکل ۱۶: تست پارسینگ فایل `cells.spaceship enginecordership7`



شکل ۱۷: تست پارسینگ فایل `in glider rle.rle`

۴.۱.۲ بهینه سازی محاسباتی با کانولوشن

در این روش باید یک ماتریس ۳ در ۳ انتخاب میکنیم به عنوان کرنل و این کرنل اینگونه محاسبه میکند که به ازای هرکدام از این اعضا این کرنل به گونه ای روی این درایه مینشیند که درایه‌ی وسطش دقیقاً روی درایه‌ی ماتریس اصلی بیفتد و سپس حاصل درایه‌ی نظیر ماتریس جدید میشود حاصل جمع ضرب درایه‌های روی هم ماتریس کرنل و ماتریس اصلی. کرنل را اینگونه انتخاب میکنیم که درایه وسط ۰ باشد و بقیه درایه‌ها ۱ باشد که ماتریس حاصل از کانولوشن بشود مجموع سلول‌های زنده در همسایگی آن سلول. این کرنل در کانستراکتور ساخته میشود:

```
1 self.neighborhood = np.ones((3, 3), np.uint)
2 self.neighborhood[1, 1] = 0
```

حالا متد update_grid_fast را پیاده میکنیم:

```
1 def update_grid_fast(self, grid):
2     if self.finite:
3         neighbors = signal.convolve2d(grid, self.neighborhood,
4             ↪ mode='same', boundary='fill')
5     else:
6         neighbors = signal.convolve2d(grid, self.neighborhood,
7             ↪ mode='same', boundary='wrap')
8     next_grid = np.zeros((self.rows, self.cols), np.uint)
9     next_grid[((grid == 1) & ((neighbors == 2) | (neighbors == 3))) |
10        ↪ ((grid == 0) & (neighbors == 3))] = 1
11     return next_grid
```

ماتریس neighbors همان ماتریس کانولوشن است که در متد signal.convolve2d، آرگومان اول همان ماتریس اصلی است که grid و آرگومان دوم ماتریس کرنل است و سوم که mode است تعیین میکند اندازه ماتریس جدید چقدر باشد که same نشان دهنده‌ی این است که اندازه ماتریس قبلی باقی می‌ماند. آرگومان آخر هم که boundary برای این است که مشخص شود wrap میکنیم یا نه که مقدار fill یعنی نداریم و wrap یعنی داریم. در آخر هم ماتریس next_grid را با شرایط چهارگانه‌ی بازی زندگی مقدار دهی میکنیم و آن را برمیگردانیم.

۲.۲ مورچه‌ی لانگتون

۱.۲.۲ پیاده سازی پایه‌ی مورچه

ابتدا کانستراکتور کلاس را میسازیم:

```
1 def __init__(self, N, ant_position=(0,0), rules={0: (1, 'R'), 1: (0,
2     ↪ 'L')}):
3     self.N = N
4     self.grid = np.zeros((N, N), np.uint)
5     self.r = ant_position[0]
6     self.c = ant_position[1]
```

```

۶     self.direction = "U"
۷     self.rules = rules

```

که در آن N نشان دهنده‌ی عرض و طول صفحه است و $ant_position$ نشان دهنده‌ی موقعیت اولیه‌ی مورچه است که به صورت دیفالت مقدار $(0, 0)$ را دارد و $Rules$ هم قوانین را مشخص میکند که مقدار پیش فرض آن همان قوانین استاندارد مورچه‌ی لانگتون است. صفحه در کانستراکتور با مقدار θ که نشان دهنده‌ی خانه‌ی سیاه است پر میشود. $self.r$ و $self.c$ به ترتیب ردیف فعلی مورچه و ستون فعلی مورچه را نشان میدهند و $self.direction$ جهت مورچه را نشان میدهد که ابتدا به سمت بالا است.

حال دو متد ساده‌ی `get_current_position` و `get_states` را پیاده میکنیم:

```

۱ def get_states(self):
۲     return self.grid
۳ def get_current_position(self):
۴     return (self.r, self.c)

```

حال متد اصلی یعنی `step` را پیاده میکنیم:

```

۱ def step(self):
۲     current_color = self.grid[self.r, self.c]
۳     if current_color == 0:
۴         self.grid[self.r, self.c] = 1
۵     else:
۶         self.grid[self.r, self.c] = 0
۷     if current_color == 0:
۸         match self.direction:
۹             case 'R':
۱0             self.direction = 'D'
۱1             self.r = (self.r + 1) % self.N
۱2             case 'L':
۱3             self.direction = 'U'
۱4             self.r = (self.r - 1) % self.N
۱5             case 'U':
۱6             self.direction = 'R'
۱7             self.c = (self.c + 1) % self.N
۱8             case 'D':
۱9             self.direction = 'L'
۲0             self.c = (self.c - 1) % self.N
۲1     else:
۲2         match self.direction:
۲3             case 'R':
۲4             self.direction = 'U'
۲5             self.r = (self.r - 1) % self.N
۲6             case 'L':
۲7             self.direction = 'D'
۲8             self.r = (self.r + 1) % self.N
۲9             case 'U':
۳0             self.direction = 'L'

```

```

۳۱         self.c = (self.c - 1) % self.N
۳۲     case 'D':
۳۳         self.direction = 'R'
۳۴         self.c = (self.c + 1) % self.N

```

طبق کد بالا ابتدا رنگ فعلی را ذخیره میکنیم و بر اساس آن خانه فعلی را رنگ میکنیم. سپس اگر رنگ قدیمی آن سیاه بود جهت چرخش مورچه باید ساعتگرد باشد پس با یک سوییچ کیس این کار را انجام میدهیم و همزمان یک خانه در جهت جدیدش آن را به حرکت در میاوریم. اگر هم رنگ قدیمی آن خانه سفید بود باید جهت چرخش مورچه پاد ساعتگرد باشد.

در آخر هم متد آپدیت که مورچه را یک استپ به حرکت در میاورد:

```

۱ def update(self):
۲     self.step()

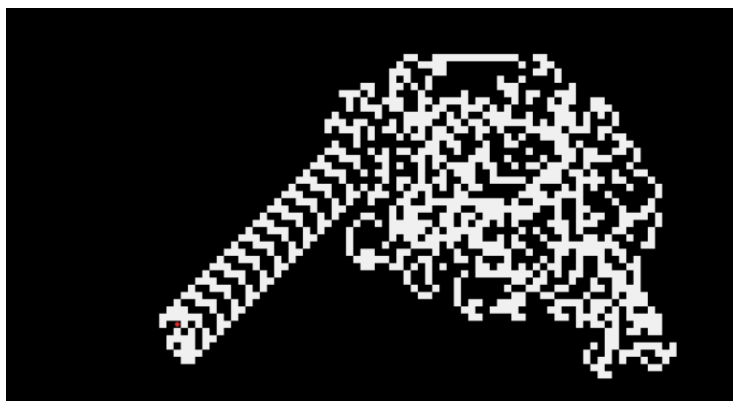
```

۲.۲.۲ شبیه سازی مورچه‌ی لانگتون و اثبات سناریو ها

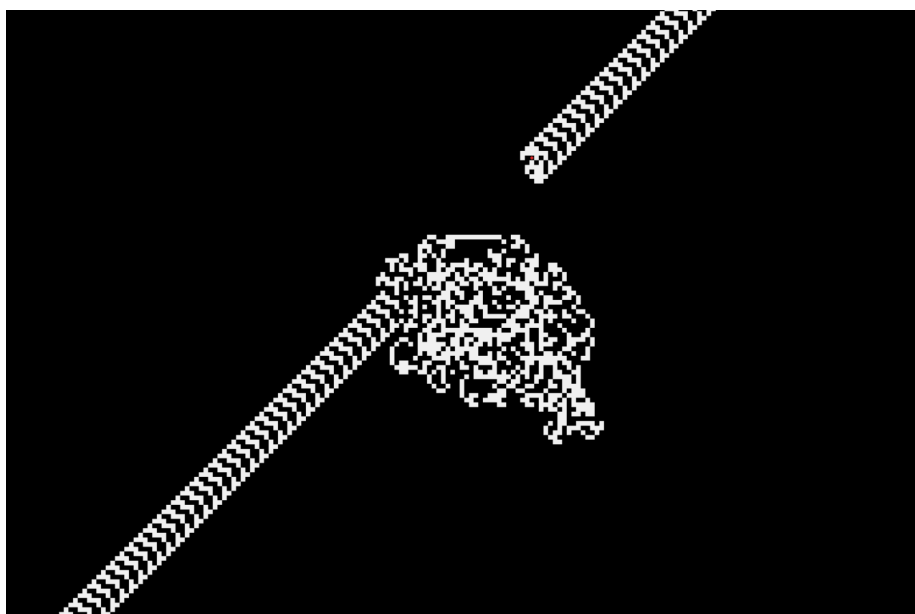
بعد شبیه سازی برنامه متوجه میشویم که این مورچه برای یه مدت طولانی رفتار آشوبناک و غیر قابل پیش بینی دارد. این رفتار برای حدود ۱۰۰۰۰ قدم ادامه میابد و بعد از آن مورچه ناگهان وارد یک الگوی تکرار شونده میشود به نام بزرگراه مورچه که در یک مسیر مورب به صورت نامتناهی حرکت میکند.



شکل ۱۸: رفتار اولیه و آشوبناک مورچه‌ی لانگتون



شکل ۱۹: رفتار تکرار شونده‌ی مورچه‌ی لانگتون (بزرگراه مورچه)



شکل ۲۰: مورچه‌ی لانگتون در حالت بزرگراه مورچه و wrap شده

۳.۲.۲ بهبود مورچه با پشتیبانی چند رنگ

بدین منظور تنها کافی است در بلاک های if متد step به جای شرایط مستقیم از شرایط نسبی استفاده کنیم.

```

۱ def step(self):
۲     current_color = self.grid[self.r, self.c]

```

```

۳ self.grid[self.r, self.c] = self.rules[current_color][0]
۴ if self.rules[current_color][1] == 'R':
۵     match self.direction:
۶         case 'R':
۷             self.direction = 'D'
۸             self.r = (self.r + 1) % self.N
۹         case 'L':
۱0            self.direction = 'U'
۱1            self.r = (self.r - 1) % self.N
۱۲         case 'U':
۱۳            self.direction = 'R'
۱۴            self.c = (self.c + 1) % self.N
۱۵         case 'D':
۱۶            self.direction = 'L'
۱۷            self.c = (self.c - 1) % self.N
۱۸ elif self.rules[current_color][1] == 'L':
۱۹     match self.direction:
۲۰         case 'R':
۲۱            self.direction = 'U'
۲۲            self.r = (self.r - 1) % self.N
۲۳         case 'L':
۲۴            self.direction = 'D'
۲۵            self.r = (self.r + 1) % self.N
۲۶         case 'U':
۲۷            self.direction = 'L'
۲۸            self.c = (self.c - 1) % self.N
۲۹         case 'D':
۳۰            self.direction = 'R'
۳۱            self.c = (self.c + 1) % self.N

```

همانطور که مشاهده میشود در خط سوم به جای اینکه مستقیماً چک کنیم ببینیم سلول سفید یا سیاه است رنگش را با رنگ جایگزینش در دیکشنری rules عوض کردیم. در خط چهارم و هجدهم به جای اینکه مستقیماً ببینیم رنگ قدیمی چه بوده تا تصمیم بگیریم ساعتگرد بچرخیم یا پادساعتگرد مستقیماً از rules چک میکنیم که به ازای رنگ قدیمی باید به چه طرفی چرخید. بدین ترتیب مورچه‌ی ما از چندین رنگ پشتیبانی میکند.



شکل ۲۱: پشتیبانی چند رنگی مورچه‌ی لانگتون

مورچه‌ی ما همچنین میتواند الگوهای بسیار زیبایی طراحی کند. برای اینکه یکی از اینها را نشان دهیم از قوانین RLLLLRLLLLRRR استفاده میکنیم. از آنجا که تعداد این قوانین ۱۲ تا هستند برای نمایش بهتر ۱۲ رنگ نیاز داریم که Palette اولیه‌ی ما تعداد کمتری رنگ داشت. بدین منظور چند رنگ به دلخواه به آن اضافه میکنیم:

```

1 PALETTE = [
2     (0, 0, 0),
3     (240, 240, 240),
4     (77, 166, 255),
5     (255, 130, 67),
6     (134, 199, 86),
7     (186, 112, 255),
8     (255, 207, 64),
9     (80, 214, 193),
10    (149, 40, 131),
11    (252, 159, 159),
12    (0, 255, 60),
13    (135, 135, 135)
14 ]

```

حالا قوانین را تحت عنوان Triangle_Rules تعریف میکنیم:

```

1 TRIANGLE_RULES = {
2     0: (1, "R"),
3     1: (2, "R"),
4     2: (3, "L"),
5     3: (4, "L"),
6     4: (5, "L"),
7     5: (6, "R"),
8     6: (7, "L"),
9     7: (8, "L"),
10    8: (9, "L"),
11    9: (10, "R"),
12    10: (11, "R"),
13    11: (0, "R")
14 }

```

حال نیاز داریم که در آرگومان‌های ورودی امکان مشخص کردن اینکه این الگو را میخواهیم اجرا کنیم را داشته باشیم. پس این خط را به تابع parse_args اضافه میکنیم.

```

1 parser.add_argument("--triangle-rules", action="store_true")

```

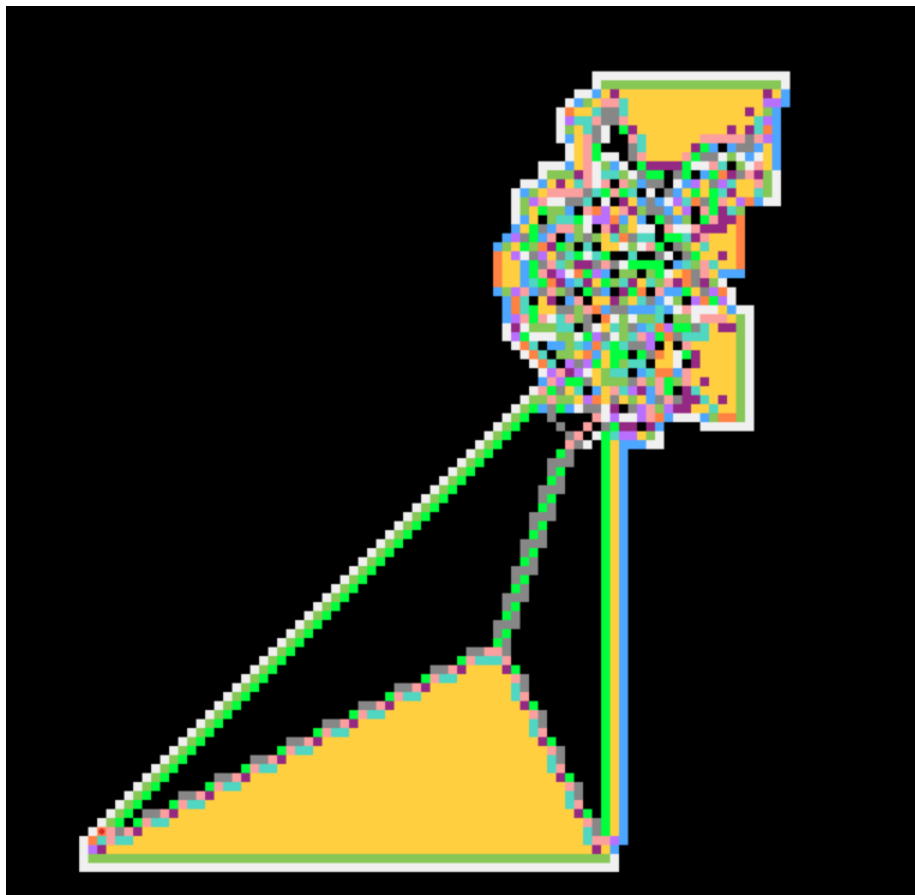
و سپس در تابع main منطق انتخاب Rule ها را اینگونه تغییر میدهیم.

```

1 if args.triangle_rules:
2     rules = TRIANGLE_RULES
3 else:
4     rules = MULTI_COLOR_RULES if args.multi_color else DEFAULT_RULES

```

بدین ترتیب اولویت با قوانین جدیدی است که ما مشخص کردیم.



شکل ۲۲: الگوی مثلث مورچه

۳.۲ گیت های منطقی در بازی زندگی

۱.۳.۲ گیت اند

هدف ما این است که با استفاده از دو گلايدر گان در مختصات (15, 12) در صورتی که ورودی ها True True بودند مربع 2x2 درست کنیم. بدین منظور اول نیاز به گلايدر گان هایی داریم که در جهت جنوبی غربی و شمالی غربی گلايدر شلیک میکنند. برای درست کردن گلايدر گانی که به سمت جنوبی غربی شلیک میکنند کافی است متدی بنویسیم که از همان مختصات گلايدر گان قبلی استفاده میکند ولی عرض مختصاتش قرینه میشود!

```
۱ def insertGliderGunSouthWest(self, index=(0, 0)):  
۲     coords = [
```

```

۳         (1, 26),
۴         (2, 24), (2, 26),
۵         (3, 14), (3, 15), (3, 22), (3, 23), (3, 36), (3, 37),
۶         (4, 13), (4, 17), (4, 22), (4, 23), (4, 36), (4, 37),
۷         (5, 2), (5, 3), (5, 12), (5, 18), (5, 22), (5, 23),
۸         (6, 2), (6, 3), (6, 12), (6, 16), (6, 18), (6, 19), (6, 24),
          ↪ (6, 26),
۹         (7, 12), (7, 18), (7, 26),
۱۰        (8, 13), (8, 17),
۱۱        (9, 14), (9, 15)
۱۲    ]
۱۳    max_col = 37
۱۴    for r, c in coords:
۱۵        c_new = max_col - c
۱۶        target_r = index[0] + r
۱۷        target_c = index[1] + c_new
۱۸        self.grid[target_r, target_c] = self.aliveValue

```

و برای تولید گلايدر گانی که به مختصات شمالی غربی شلیک میکند باید هم عرض و هم طول مختصات گلايدر گان استاندارد اولیه مان را قرینه کنیم:

```

۱    def insertGliderGunNorthWest(self, index=(0, 0)):
۲        coords = [
۳            (1, 26),
۴            (2, 24), (2, 26),
۵            (3, 14), (3, 15), (3, 22), (3, 23), (3, 36), (3, 37),
۶            (4, 13), (4, 17), (4, 22), (4, 23), (4, 36), (4, 37),
۷            (5, 2), (5, 3), (5, 12), (5, 18), (5, 22), (5, 23),
۸            (6, 2), (6, 3), (6, 12), (6, 16), (6, 18), (6, 19), (6, 24),
          ↪ (6, 26),
۹            (7, 12), (7, 18), (7, 26),
۱۰           (8, 13), (8, 17),
۱۱           (9, 14), (9, 15)
۱۲        ]
۱۳        max_col = 37
۱۴        max_row = 9
۱۵        for r, c in coords:
۱۶            c_new = max_col - c
۱۷            r_new = max_row - r
۱۸            target_r = index[0] + r_new
۱۹            target_c = index[1] + c_new
۲۰            self.grid[target_r, target_c] = self.aliveValue

```

حالا که این دو گلايدر گان را ساختیم باید مختصاتی پیدا کنیم که این دو را آنجا تولید کنیم تا در مختصات هدف ما مربع 2x2 بسازند. برای اینکار در فایل `pygame_viewer` و در تابع `grid_to_surface` این خطوط را میزاریم تا مختصات هدفمان را رنگ کنیم:

```

۱    for x in (12, 13):
۲        for y in (15, 16):

```

```

۳     if cells[x, y] == 0:
۴         rgb[x, y] = (0,0,255)
۵     else:
۶         rgb[x,y] = (255,0,0)

```

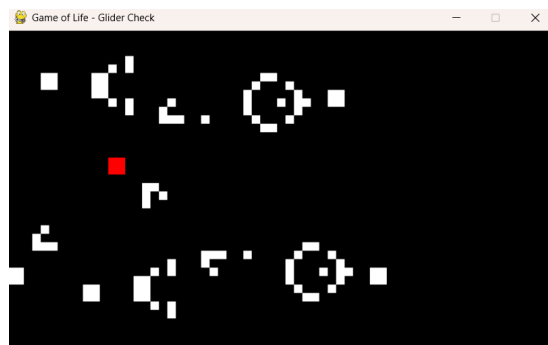
طبق این خطوط سلول های مرده ای این مختصات آبی و سلول های زنده ای این مختصات قرمز میشوند.

دلیل اینکه به جای ردیف های ۱۵ و ۱۶، ردیف های ۱۲ و ۱۳ و به جای ستون های ۱۳ و ۱۲، ستون های ۱۵ و ۱۶ رنگ شدند این است که ابتدای این تابع شاهد این خط بودیم `cells = np.swapaxes(grid, 0, 1)` و این خط جای ستون و ردیف ها را عوض کرده. حالا در فایل `find_places_and_steps` مختصات مختلف این گلايدر گان ها را تست میکنیم تا بلخره در نقطه ای رنگ شده مربع بسازند.

```

۱ import conway
۲ from pygame_viewer import run_pygame_life
۳ N = 65
۴ CELL_SCALE = 10
۵ def main():
۶     life = conway.GameOfLife(N, finite=True, fastMode=True)
۷     life.insertGliderGunSouthWest((2, 4))
۸     life.insertGliderGunNorthWest((25,9))
۹     run_pygame_life(life, cell_scale=CELL_SCALE, fps=50,
۱۰                     max_frames=300, title="Game of Life - Glider Check")
۱۱ if __name__ == "__main__":
    ^^^main()

```



شکل ۲۳: پیدا کردن مختصات گلايدر گان ها برای ساختن مربع در مختصات هدف

حالا که مختصات قرار دادن گلايدر گان ها را پیدا کردیم باید دقیقاً مشخص کنیم در چندمین گام این مربع ساخته میشود که بتوانیم گیت اند را بسازیم. برای این منظور کافی است در حلقه ای `while not finished` این خطوط از کد را مینویسیم:

```

۱ if cells[15:17, 12:14].sum() == 4:
۲     finished = True

```

مطابق این خطوط، برنامه به محض اینکه مربع 2x2 در این مختصات ساخته شد متوقف میشود. حالا نیاز داریم تعداد گام ها را چاپ کنیم که کافی است قبل `pygame.quit()` تعداد فریم ها را چاپ کنیم: `.print(frame)`.
 با اجرا کردن فایل `find_places_and_steps` خروجی عدد ۶۰ را نشان میدهد.
 حالا تمام اطلاعات لازم برای پیاده سازی متد های مربوط به گیت And در فایل `logic_gates` را داریم.

```

1 def setup_and_gate(self, grid_size=65, input_a_present=False,
2   ↪ input_b_present=False):
3     life = GameOfLife(grid_size, finite=True, fastMode=True)
4     if input_a_present:
5         life.insertGliderGunSouthWest((2,4))
6     if input_b_present:
7         life.insertGliderGunNorthWest((25,9))
8     return life
9
10 def run_and_gate(self, input_a_present, input_b_present):
11     life = self.setup_and_gate(input_a_present=input_a_present,
12   ↪ input_b_present=input_b_present)
13     for _ in range(60):
14         life.evolve()
15     if life.getGrid()[15:17, 12:14].sum() == 4:
16         return True
17     else:
18         return False

```

و فایل `test_logic_gate_and` اینگو مینویسیم:

```

1 from logic_gates import GliderLogicGates
2 from pygame_viewer import run_pygame_life
3 N = 65
4 CELL_SCALE = 10
5 def main():
6     input_a = True
7     input_b = True
8     LogicGate = GliderLogicGates()
9     life = LogicGate.setup_and_gate(N, input_a, input_b)
10    run_pygame_life(life, cell_scale=CELL_SCALE, fps=18,
11   ↪ max_frames=300, title="Game of Life - Gate And")
12    print(LogicGate.run_and_gate(input_a, input_b))
13
14 if __name__ == "__main__":
15     ^__Imain()

```

۲.۳.۲ گیت نات

برای پیاده سازی این گیت باید مختصات دو گلايدر گان برخورد کننده را جوری پیدا کنیم که گلايدر های همدیگر را نابود کنند. یکی از این گلايدر گان ها نقش ورودی و دیگری نقش گلايدر

گان کنترل را بازی خواهند کرد. در فایل `find_places_and_steps` حالات مختلف را بررسی میکنیم تا مختصات مطلوب را پیدا کنیم:

```

1 import conway
2 from pygame_viewer import run_pygame_life
3 N = 65
4 CELL_SCALE = 10
5 def main():
6     life = conway.GameOfLife(N, finite=True, fastMode=True)
7     life.insertGliderGunNorthWest((31,15))
8     life.insertGliderGunSouthWest((10,16))
9     run_pygame_life(life, cell_scale=CELL_SCALE, fps=50,
10                     max_frames=300, title="Game of Life - Glider Check")
11 if __name__ == "__main__":
12     main()

```



شکل ۲۴: پیدا کردن مختصات گلايدر گان ها برای ساختن گيت نات

حالا که مختصات مطلوب گلايدرگان ها را پیدا کردیم، وقت آن است شرایط خروجی `True` را مشخص کنیم. طبق استدلال های شهودی گلايدر های گلايدر گان کنترل به مربع دو در دو در مختصات (2, 0) میرسند. پس کافی است در فایل `pygame_viewer` و در حلقه `While` `finished` این دو خط را بنویسیم:

```

1 if cells[2:4 , 0:2].sum() > 0:
2     finished = True

```

و قبل `pygame.quit()`، تعداد فریم ها را چاپ کنیم: `print(frame)`.
خواهیم دید عدد چاپ شده بعد از اجرای `find_places_and_steps` به ازای فقط گلايدر

گان کنترل عدد ۱۲۹ میباشد. حالا هر آنچه برای پیاده سازی گیت نات نیاز داریم را داریم. متد های مربوط به گیت نات در فایل logic_gates را کامل میکنیم:

```

1 def setup_not_gate(self, grid_size=65, input_a_present=False):
2     life = GameOfLife(grid_size,finite=True, fastMode=True)
3     if input_a_present:
4         life.insertGliderGunSouthWest((10,16))
5         life.insertGliderGunNorthhhWest((31,15))
6     return life
7
8 def run_not_gate(self, input_a_present):
9     life = self.setup_not_gate(input_a_present=input_a_present)
10    for _ in range(129):
11        life.evolve()
12    if life.getGrid()[2:4 , 0:2].sum() > 0:
13        return True
14    else:
15        return False

```

و در آخر فایل test_logic_gate_not را مینویسیم:

```

1 from logic_gates import GliderLogicGates
2 from pygame_viewer import run_pygame_life
3 N = 65
4 CELL_SCALE = 10
5 def main():
6     input_a = True
7     LogicGate = GliderLogicGates()
8     life = LogicGate.setup_not_gate(N,input_a)
9     run_pygame_life(life, cell_scale=CELL_SCALE, fps=50,
10        ↪ max_frames=300, title="Game of Life - Glider Check")
11     print(LogicGate.run_not_gate(input_a))
12
13 if __name__ == "__main__":
14     ^^Imain()

```